

Week 5 - Task 2: Azure IAM and Cryptography

Part 1: Azure IAM Role Assignment

Objective: To create a custom role with read-only permissions on Azure Blob Storage and assign that role to a user.

Steps:

1. Azure CLI Login:

- Executed az login to authenticate with my Azure account.
- This allowed the Azure CLI to interact with my Azure subscription.

2. Role Definition Creation:

- Created a JSON file named role.json with the following content:

JSON

```
{  
  "Name": "ReadBlobs",  
  "Description": "Read blob data.",  
  "Actions": ["Microsoft.Storage/storageAccounts/blobServices/containers/blobs/read/content"],  
  "AssignableScopes":  
    ["/subscriptions/YOUR_SUBSCRIPTION_ID/resourceGroups/myResourceGroup"]  
}
```

- Replaced YOUR_SUBSCRIPTION_ID with my actual Azure subscription ID.

- Executed az role definition create --role-definition role.json to create the custom role named "ReadBlobs". This role grants read-only access to blob content within the specified resource group.

3. User Object ID Retrieval:

- Executed az ad user list --query "[?userPrincipalName=='dipukumarbth5840@gmail.com'].objectId" -o tsv to retrieve the object ID of the user dipukumarbth5840@gmail.com.
- Copied the returned object ID.

4. Role Assignment:

- Executed the following command to assign the "ReadBlobs" role to the user within the "myResourceGroup" resource group:

Bash

```
az role assignment create --assignee dipukumarbth5840@gmail.com --role "ReadBlobs" --scope "$(az group show --name myResourceGroup --query id --output tsv)"
```

- This command granted the specified user the permissions defined in the "ReadBlobs" role, scoped to the "myResourceGroup" resource group.

```
az role assignment create --assignee sp_name --role Reader --scope /subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/MyResourceGroup/providers/Microsoft.Compute/virtualMachines/MyVm
Create role assignment to grant the specified assignee the Reader role on an Azure virtual machine.

az role assignment create --role Owner --scope /subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/MyResourceGroup/providers/Microsoft.Storage/storageAccounts/MyStorageAccount --assignee "John.Doe@Contoso.com" --description "Role assignment foo to check on bar" --condition "@Resource[Microsoft.Storage/storageAccounts/blobServices/containers:Name] stringEquals 'foo'" --condition-version "2.0"
Create role assignment for an assignee with description and condition.

az role assignment create --assignee-object-id 00000000-0000-0000-0000-000000000000 --assignee-principal-type ServicePrincipal --role Reader --scope /subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/MyResourceGroup --name 00000000-0000-0000-0000-000000000000
Create role assignment with your own assignment name.
```

Part 2: Cryptography with OpenSSL

Objective: To encrypt and decrypt a file using AES-256, generate RSA key pairs, and verify file integrity using SHA-256 hashing.

Steps:

1. File Creation:

- Created a text file named mysecret.txt with the content "secret text" using the command `echo "secret text" > mysecret.txt`.

2. AES-256 Encryption:

- Encrypted mysecret.txt using AES-256 with the command `openssl enc -aes-256-cbc -salt -in mysecret.txt -out secret.enc`.
- Entered and verified a password when prompted.

3. AES-256 Decryption:

- Decrypted secret.enc using the same password with the command `openssl enc -aes-256-cbc -d -in secret.enc -out secret_plain.txt`.

4. RSA Key Pair Generation:

- Generated a 2048-bit RSA private key using `openssl genrsa -out private.pem 2048`.
- Generated the corresponding public key from the private key using `openssl rsa -in private.pem -out public.pem -outform PEM -pubout`.

5. SHA-256 Hashing (Original File):

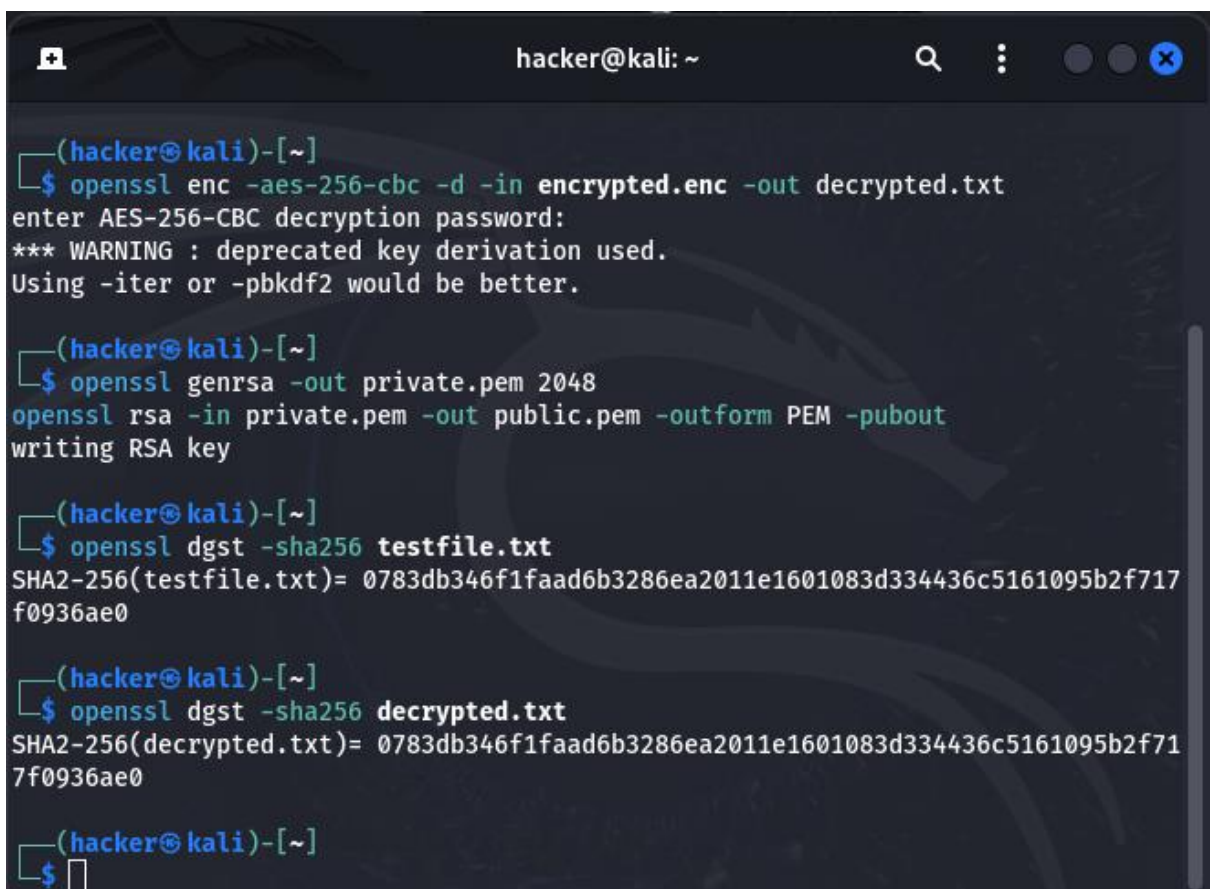
- Calculated the SHA-256 hash of mysecret.txt using openssl dgst -sha256 mysecret.txt.
- Copied the resulting hash value.

6. SHA-256 Hashing (Decrypted File):

- Calculated the SHA-256 hash of secret_plain.txt using openssl dgst -sha256 secret_plain.txt.
- Copied the resulting hash value.

7. Hash Verification:

- Compared the two hash values. They matched, confirming the integrity of the decrypted file.



```

hacker@kali: ~
(hacker@kali)-[~]
$ openssl enc -aes-256-cbc -d -in encrypted.enc -out decrypted.txt
enter AES-256-CBC decryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

(hacker@kali)-[~]
$ openssl genrsa -out private.pem 2048
openssl rsa -in private.pem -out public.pem -outform PEM -pubout
writing RSA key

(hacker@kali)-[~]
$ openssl dgst -sha256 testfile.txt
SHA2-256(testfile.txt)= 0783db346f1faad6b3286ea2011e1601083d334436c5161095b2f717f0936ae0

(hacker@kali)-[~]
$ openssl dgst -sha256 decrypted.txt
SHA2-256(decrypted.txt)= 0783db346f1faad6b3286ea2011e1601083d334436c5161095b2f717f0936ae0

(hacker@kali)-[~]
$

```

8.